

Aula 1 — Arquitetura do PIC18

O que há dentro do chip, e o que isso muda no seu programa

Microcontroladores — DENE/UFMT

OBJETIVOS

- Enumerar os blocos internos do PIC18F4550 e localizar os pinos no mapa de memória de dados.
- Justificar, a partir da separação Harvard, por que a RAM se esgota antes da Flash num projeto que cresce.
- Calcular o tempo de execução a partir da frequência do oscilador e identificar o erro de fator quatro que atravessa todo o semestre.
- Estimar o menor intervalo de tempo que este dispositivo consegue *garantir*, e distinguir velocidade de previsibilidade.
- Escrever em C sobre um registrador de porta sem alterar os bits vizinhos, e escolher tipos inteiros cujo tamanho esteja escrito no código.

1 Toda arquitetura decide coisas por você

Um microcontrolador não é uma folha em branco. Antes de vocês escreverem a primeira linha de código, alguém já decidiu quanto o chip conta por segundo, quanta memória existe e como ela é dividida, e em que estado os pinos acordam quando a placa é ligada. Esse conjunto de decisões é o que se chama de **arquitetura**, e ela tem uma propriedade incômoda: não se negocia. O programa é que tem de caber nela.

Isso não é particularidade do PIC18. ESP32, STM32, RISC-V — todos têm arquitetura, todos decidem exatamente as mesmas coisas, e todos decidem de forma diferente. Trocar de plataforma não elimina o problema; troca os números. É por isso que vale aprender a **fazer as perguntas**, e não decorar as respostas de um dispositivo específico.

Na prática, a arquitetura chega até o trabalho de vocês por três portas:

O que a arquitetura decide	O que isso custa a vocês
Como a memória é organizada, e o que está ligado atrás de cada endereço	Explica por que uma atribuição comum aciona um pino, e determina qual recurso vai acabar primeiro quando o projeto crescer
Quanto tempo custa cada instrução	Todo cálculo de temporização do semestre parte daí: piscar um LED, gerar um PWM, fixar uma taxa de comunicação. Errar esse número invalida todos os outros de uma vez
O que já está configurado antes da primeira instrução executar	Produz falhas que se parecem com defeito de programa, e que nenhuma releitura do código revela

NOTA

Guardem a terceira linha. Ela é a que mais custa tempo de laboratório, e quase nunca por dificuldade conceitual — por descuido de configuração. O sintoma aponta para o lugar errado.

1.1 O percurso de hoje

A aula segue uma linha só, e cada trecho dela desemboca num sintoma que vocês vão encontrar na bancada daqui a trinta minutos.

O que a seção estabelece	O que isso explica na bancada
2 Quais peças existem dentro do encapsulamento	Por que um mesmo pino faz três coisas diferentes

3	Os pinos são posições de memória	Por que escrever <code>LATD = 0xFF</code> acende oito LEDs
4	Programa e dados moram em espaços separados	Por que o projeto vai esbarrar na RAM antes da Flash
5	Não há sistema operacional entre você e o pino	Por que o LED pisca fora do ritmo — e quanto tempo real dá para conseguir
6	Alguém configurou o chip antes de você	Por que um botão na porta B parece não funcionar
7	O C daqui não é o C de algoritmos	Por que acender um LED pode desligar o aquecedor

NOTA

A seção 8 reúne o restante da arquitetura — pilha, contador de programa, paralelismo, banqueamento — como **contexto**. Ela explica por que as coisas são assim, não o que fazer na bancada, e reaparece no seminário comparativo.

2 O que há dentro do encapsulamento

Um microcontrolador é um computador inteiro numa pastilha: processador, memória, fonte de tempo e periféricos, ligados por barramentos internos. A diferença para o computador de vocês não é conceitual, é de escala e de finalidade — e a consequência dessa diferença ocupa o resto da aula.

2.1 O inventário

Bloco	PIC18F4550
Núcleo	8 bits, arquitetura Harvard, multiplicador por hardware
Fonte de tempo	Oscilador a cristal externo, com multiplicador e divisores programáveis
Memória de programa	32 KB de Flash — 16 384 instruções de uma palavra
Memória de dados	2 048 bytes de RAM estática — os GPR. Os registradores de periférico, os SFR, ocupam o topo do mesmo espaço de endereços
Memória não volátil de dados	256 bytes de EEPROM
Frequência máxima do dispositivo	48 MHz, equivalentes a 12 MIPS
Frequência do núcleo nesta bancada	16 MHz, equivalentes a 4 MIPS
Portas de entrada e saída	35 pinos no encapsulamento de 40 vias
Conversor analógico-digital	10 bits, 13 canais
Comparadores analógicos	Dois, com referência programável
Temporizadores	Quatro: um de 8 bits e três de 16
Modulação	Dois módulos de captura, comparação e PWM
Comunicação	Serial assíncrona, síncrona mestre-escravo e controlador USB

As duas linhas de frequência não são um erro de digitação. A diferença entre elas é assunto da seção 5.

NOTA

Vale medir a escala uma vez, para calibrar a intuição de quem vem de programação de computadores: a memória de dados deste chip é de 2 KB. Uma única fotografia do celular tem alguns milhões de vezes isso. Não estamos diante de um computador pequeno; estamos diante de outra categoria de máquina, que existe para outra finalidade.

2.2 Trinta e cinco pinos, e quase nenhum faz uma coisa só

O encapsulamento tem 40 vias. Cinco são alimentação e referência de tensão; as outras 35 são pinos de entrada e saída, distribuídos em cinco portas — A, B, C, D e E. Mas esse número é o do papel. Nesta bancada, dois deles estão ocupados pelo cristal e um é o pino de reinicialização, de modo que sobram 32.

O fato mais importante desta seção não é a contagem. É que **o silício tem mais funções do que pinos**, e a saída da Microchip para isso foi multiplexar: o mesmo pino físico é ligado a vários blocos internos, e um bit de configuração decide qual deles fala com o mundo naquele momento.

Pino	Funções que disputam esse pino
RB4	Entrada e saída digital; canal analógico AN11
RB6, RB7	Entrada e saída digital; canais analógicos; linhas do gravador
RC1	Entrada e saída digital; entrada do oscilador do Timer1; saída do módulo CCP2
RC6, RC7	Entrada e saída digital; transmissão e recepção da serial
RE3	Entrada digital; pino de reinicialização externa

CONCEITO

Multiplexação de pinos é a razão de existirem os bits de configuração, e a razão de um pino poder “não funcionar” com um programa correto: ele está funcionando, só que como outra coisa.

Guardem a formulação, porque ela vai reaparecer no laboratório com sinal trocado: **quando um pino não responde, a primeira hipótese não é o código — é que ele está atendendo a outro dono.**

Isso também explica um limite prático do projeto do semestre. Usar a serial custa RC6 e RC7; usar PWM custa RC1 ou RC2; usar entradas analógicas custa pinos da porta A e da porta B. O orçamento de pinos é tão real quanto o de memória, e aparece antes dele.

3 Os pinos são posições de memória

Na oficina, uma atribuição a uma variável de aparência perfeitamente comum acendeu uma barra de LEDs. Isso não é açúcar sintático da biblioteca: `LATD` é literalmente um endereço de memória, e há hardware ligado atrás dele.

CONCEITO

No vocabulário da Microchip, **toda** posição de memória de dados é um registrador — um *file register*. Não existe um banco de registradores separado da RAM, como em ARM ou RISC-V. `MOVWF 0x20` e `MOVWF LATD` são a mesma instrução, com endereços diferentes.

Esses file registers se dividem em duas famílias, e as duas siglas reaparecem no resto do semestre — no mapa do ligador, na listagem e no datasheet:

GPR *general purpose register*, registrador de uso geral. É a RAM comum: suas variáveis. Não há hardware atrás.

SFR *special function register*, registrador de função especial. Cada um é a interface de um bloco interno — uma porta, um temporizador, o conversor analógico-digital.

O que separa as duas famílias não é a instrução usada nem o tipo de memória — é o que está ligado atrás do endereço. Escrever num GPR apenas guarda um número; escrever num SFR **muda o comportamento do chip**.

Faixa	Nome	O que é
0x000 – 0x7FF	GPR	RAM de uso geral. Suas variáveis. Nada de hardware atrás

0x800 –	—	Não implementado. Lê zero; escrita é descartada
0xF5F		
0xF60 –	SFR	Registradores de função especial: TRIS , LAT , PORT , ADCON , T0CON ,
0xFFF		TXREG , STATUS , WREG , BSR

É por isso que não existe, nesta disciplina, uma função `escreverNoPino()` que precise ser aprendida. A operação de escrever num pino é a operação de escrever na memória, que vocês já sabem fazer. O que muda é o endereço.

NOTA

Metade da memória de dados deste dispositivo é fisicamente compartilhada com o controlador USB. Com o módulo desabilitado — o caso deste projeto — a região fica disponível como memória de uso geral. É peculiaridade do dispositivo, não da família.

3.1 Três registradores por porta

Cada porta é controlada por três SFR, e confundi-los é a fonte mais comum de erro no primeiro roteiro.

Registra- dor	Papel
TRISx	Direção do pino: 1 é entrada, 0 é saída
LATx	O que você escreve — o valor mantido no latch de saída
PORTx	O que você lê — o estado elétrico do pino

A distinção entre LAT e PORT parece redundante e não é: um registrador guarda o que você **mandou**, o outro mede o que o pino **está**. Os dois divergem sempre que o mundo externo discorda do programa — um curto, um pino sobrecarregado, uma saída em conflito com outro dispositivo. A seção 7 mostra o caso em que essa divergência corrompe um bit vizinho.

ATENÇÃO

No *reset*, todo pino nasce como entrada e LAT é indefinido. Habilitar a saída em TRIS antes de escrever um estado seguro em LAT faz o pino comandar o atuador com lixo, por um breve intervalo.

Com um LED isso é invisível. Com um cooler ou uma resistência de aquecimento, não é. Daí a regra: **LAT antes de TRIS**.

Na aula de laboratório, vocês vão inverter a ordem deliberadamente e pressionar o *reset* várias vezes, prestando atenção no cooler. Anotem agora o que esperam ouvir.

4 Programa e dados moram em espaços separados

O dispositivo tem 32 KB de Flash e 2 KB de RAM — uma proporção de dezesseis para um. O laboratório acrescenta uma camada por semana ao mesmo código, e a experiência de outros semestres é consistente: a memória de programa raramente é o limite deste projeto; os 2 KB de RAM são.

CONCEITO

Arquitetura Harvard, no sentido operativo desta disciplina: memória de programa e memória de dados com espaços de endereçamento distintos e barramentos próprios, podendo ter larguras diferentes. Aqui, 16 bits para programa e 8 para dados.

A consequência imediata é que os dois números são independentes. Uma cadeia de texto constante grande não rouba espaço das variáveis — são dois orçamentos, e cada um estoura por conta própria.

ATENÇÃO

Buffers de comunicação, vetores de média móvel e cadeias de texto consomem RAM rapidamente. Um vetor de 64 amostras de 16 bits, sozinho, já toma 128 bytes — mais de 6% do total.

A boa notícia é que o estouro é reportado pelo **ligador**: é erro de compilação, não falha em execução.

A separação também cobra um preço. Como `const` mora na Flash e a Flash não é endereçável como dado, ler uma constante exige instruções específicas de leitura de tabela (`TBLRD`), não um simples acesso à memória. O compilador cuida disso, mas o custo em ciclos é real e aparece na listagem.

4.1 O que o compilador coloca em cada lugar

Construção em C	Onde reside	Observação
Código das funções	Flash	Consome memória de programa
<code>const</code> e literais de texto	Flash	Acesso por leitura de tabela
Variáveis globais e estáticas	RAM	Existem durante toda a execução
Variáveis locais	RAM	Endereços fixos, atribuídos pelo compilador
Variáveis de tratamento	RAM, com <code>volatile</code>	Vide Aula 9
Dados persistentes	EEPROM	Escrita explícita; vide Aula 10

4.2 Onde a variável mora**NOTA**

Escrever `int x;` num programa de computador é pedir espaço a um sistema operacional que decide, em tempo de execução, onde a variável vai morar. O endereço muda a cada execução, é virtual, e o programador nunca precisa saber qual é.

Aqui não há sistema operacional, não há memória virtual e não há tempo de execução para decidir nada: o compilador escolhe um endereço físico fixo — digamos `0x022` — e é lá que `x` vive, sempre, em todas as execuções. Essa é a diferença que organiza todas as outras.

Conceito	No computador	No PIC18
Variável local	Nasce e morre num quadro de pilha; endereço só existe em execução	Endereço fixo dado pelo compilador, reaproveitado entre funções que não coexistem
Pilha	Uma só, em RAM, guarda retorno, argumentos e locais	Só endereços de retorno, em silício dedicado, 31 níveis; não consome RAM
Ponteiro inválido	Falha de segmentação; o programa morre	Nada acontece — ou um pino comuta, ou um periférico muda de modo
Alocação dinâmica	Rotineira	Praticamente inexistente; 2 KB não comportam heap
Memória disponível	Gigabytes	2 048 bytes, verificados pelo ligador em tempo de compilação

ATENÇÃO

Duas consequências merecem ser guardadas desde já.

A primeira é a **não reentrância**: se as variáveis locais têm endereço fixo, a mesma função chamada do laço principal e de um tratador de interrupção usará as mesmas posições de memória nas duas invocações — e uma sobrescreverá a outra. Isso é retomado na Aula 9.

A segunda é a **ausência de rede de proteção**. Num computador, o erro de memória se manifesta como um programa que morre; aqui, como um programa que continua rodando e faz outra coisa. O sintoma não aponta para a causa.

O sistema operacional que não está aqui decidia **onde**. Ele também decidia **quando** — e é disso que trata a seção seguinte.

5 Não há sistema operacional entre você e o pino

Num computador, entre o seu `write()` e o dispositivo há um driver, um escalonador, filas e outros processos concorrendo. Vocês não escolhem o instante em que a operação acontece; escolhem apenas a ordem em que pedem. Aqui, entre a instrução e o pino não há absolutamente nada. O programa é o único que roda, e o instante em que o pino comuta é decidido pela instrução que vocês escreveram.

Isso é uma promessa e uma conta. A promessa é o controle de tempo. A conta é que todo tempo gasto é tempo cobrado de vocês: não existe ninguém para preencher as lacunas.

5.1 O ciclo de instrução

CONCEITO

Cada ciclo de instrução consome **quatro** ciclos do oscilador:

$$T_{cy} = \frac{4}{f_{osc}}$$

Com o núcleo a 16 MHz, como nesta bancada:

$$T_{cy} = \frac{4}{16 \cdot 10^6} = 250 \text{ ns}$$

ou seja, 4 milhões de instruções por segundo. Uma rotina de 120 instruções sem desvios executa em cerca de 30 μ s.

ATENÇÃO

Confundir frequência do oscilador com frequência de instrução é um erro frequente. Ele reaparece nos encontros de temporizadores e de modulação, porque **todos** os divisores partem de T_{cy} , nunca de f_{osc} . Um erro de fator quatro num período é quase sempre este.

5.2 Quanto tempo real dá para conseguir

Com $T_{cy} = 250$ ns, dá para responder à pergunta em números:

Operação	Custo
Uma instrução simples: <code>LATDbits.LATD0 = 1</code>	250 ns
Um desvio	500 ns
Laço mínimo que comuta um pino: comutação mais desvio	750 ns por transição, ou onda de ≈ 667 kHz
Do evento até a primeira instrução do tratador de interrupção	$\approx 1 \mu$ s
Incerteza desse atraso (a instrução em curso precisa terminar)	até 250 ns
Uma conversão analógico-digital completa	dezenas de μ s

A linha que mais importa é a penúltima. O atraso até o tratador não é apenas curto: ele é **conhecido**, e varia dentro de uma janela de um único ciclo de instrução. É essa propriedade — e não a velocidade — que permite construir um controle de temperatura cujo comportamento se possa prever.

CONCEITO

Rápido e previsível são propriedades diferentes.

O computador de vocês é milhares de vezes mais rápido que este chip e, ainda assim, não garante resposta em 10 μ s. O escalonador do sistema operacional pode atender numa volta em microssegundos e na volta seguinte em milissegundos, porque outro processo tinha prioridade. Para controle, o que conta não é o caso médio; é o pior caso.

O PIC18F4550 é lento e **garante**. Essa troca é a razão de existirem microcontroladores num mundo que já tem processadores muito melhores — e é a justificativa da disciplina inteira.

EXPERIMENTO

Quem já usou Arduino tem aqui uma medida instrutiva a fazer na bancada. `digitalWrite()` resolve em tempo de execução qual porta e qual bit acionar; a escrita direta no registrador resolve isso em tempo de compilação. Comutem um pino num laço apertado das duas maneiras e comparem as frequências no osciloscópio. A diferença é de ordens de grandeza, e é a mesma abstração que torna o Arduino agradável de usar e impróprio para temporização fina.

5.3 De onde vem o clock

O PIC18F4550 tem uma árvore de clock desproporcional para um dispositivo de 8 bits, e a razão é o USB — periférico que este projeto não usa, mas cuja existência todos pagam na configuração.

O controlador USB exige exatamente 48 MHz, gerados por um multiplicador interno de razão fixa que precisa receber, na entrada, exatamente 4 MHz. Daí a cadeia:

1. o cristal externo entra no dispositivo;
2. um primeiro divisor, `PLLDIV`, reduz essa frequência aos 4 MHz exigidos;
3. o multiplicador eleva esses 4 MHz a **96 MHz**;
4. um divisor fixo por dois deriva daí os 48 MHz que o USB consome;
5. um segundo divisor, `CPUDIV`, divide os mesmos 96 MHz e define a frequência entregue ao processador.

O último passo é o que separa as duas linhas de frequência da tabela da seção 2. Nesta bancada a divisão é por seis, e o núcleo recebe **16 MHz**, enquanto o ramo do USB permanece em 48 MHz. As duas frequências coexistem no mesmo instante; a que interessa a vocês é a do núcleo.

DIVERGÊNCIA

Quem decide isso na XM118 não é o programa de vocês.

A placa vem com um **bootloader** gravado, e é ele que fixa as palavras de configuração — inclusive `PLLDIV`, `CPUDIV` e `FOSC`. Um bloco `#pragma config` escrito na aplicação compila sem erro, é gravado sem reclamação e **não tem efeito nenhum**: os bits já estão definidos e o bootloader é o dono deles.

A consequência prática para o laboratório é uma linha só:

```
#define _XTAL_FREQ 16000000UL
```

Se esse valor estiver em `48000000UL`, o compilador calculará os laços de `__delay_ms` para uma CPU três vezes mais rápida do que a real, e todos os atrasos sairão **três vezes mais longos**. É por isso que a oficina pede cronômetro: vinte piscadas medidas são a evidência mais barata de que o clock está correto.

TAREFA

Na aula prática, com `_XTAL_FREQ` correto, o período medido de um `__delay_ms(500)` dobrado bate com 1 s? Troque deliberadamente para `48000000UL`, meça de novo e confirme o fator três. Leve o cronômetro.

6 Alguém configurou o chip antes de você

Um botão ligado à porta B não é detectado. O mesmo código, com o botão movido para a porta D, funciona. O programa está correto — e está mesmo.

A causa já foi antecipada na seção 2: os pinos da porta B são multiplexados com canais do conversor analógico-digital, e um bit de configuração, `PBADEN`, decide se eles nascem analógicos ou digitais. O padrão **não** é digital. Enquanto o pino estiver como entrada analógica, o buffer digital está desligado e a leitura devolve algo que não corresponde ao botão. A porta D não tem canais analógicos; por isso o mesmo código funciona lá.

CONCEITO

As palavras de configuração são gravadas junto com o programa e definem o comportamento do dispositivo **antes da primeira instrução executar**. Nenhum erro nelas é detectado pelo compilador, e nenhum se manifesta como erro de compilação: todos se manifestam como comportamento estranho em execução.

Bit	Por que ele aparece nesta lista
<code>XINST</code>	Habilita o conjunto estendido de instruções, não suportado pelo compilador usado. Ligado por engano, o programa executa lixo. Deve permanecer desabilitado
<code>LVP</code>	Programação em baixa tensão. Habilitada, mantém um pino da porta B reservado, que deixa de funcionar como entrada e saída comum — e um ruído nesse pino pode colocar o dispositivo em modo de programação
<code>WDT</code>	Cão de guarda. Habilitado sem que o programa o alimente, provoca reinicializações periódicas que se manifestam como “o programa reinicia sozinho”
<code>PBADEN</code>	Define se pinos da porta B iniciam como analógicos ou digitais. O padrão não é digital: um botão ligado a esses pinos parece não funcionar até que isso seja corrigido
<code>MCLRE</code>	Define se o pino de reinicialização externa é reset ou entrada digital. Alterá-lo sem necessidade pode impedir a gravação
<code>FOSC</code> , <code>PLLDIV</code> , <code>CPUDIV</code>	Definem a frequência efetiva. Errados, corrompem toda a temporização e a comunicação serial

```
/* Bloco de configuracao tipico de uma placa SEM bootloader,
   ajustado para reproduzir esta bancada: nucleo a 16 MHz.
   Na XM118 estes valores sao fixados pelo bootloader e as
   diretivas abaixo, se escritas na aplicacao, sao ignoradas. */
#pragma config PLLDIV   = 5           /* 20 MHz / 5 = 4 MHz no PLL      */
#pragma config CPUDIV   = OSC4_PLL6  /* 96 MHz / 6 = 16 MHz no nucleo */
#pragma config FOSC     = HSPLL_HS
#pragma config WDT      = OFF         /* ligado somente na Aula 10 */
#pragma config LVP      = OFF         /* libera o pino da porta B */
#pragma config XINST    = OFF         /* obrigatorio com XC8 */
#pragma config PBADEN   = OFF         /* porta B digital no reset */
#pragma config MCLRE    = ON
```

ATENÇÃO

Vale registrar o método por trás desta seção, mais do que a lista. Nenhum desses bits é conceitualmente difícil; todos produzem sintomas que **parecem** defeitos de programa. Quando um código correto se comporta de maneira inexplicável, verifique a configuração antes de reescrever o código.

7 O C que a bancada exige

NOTA

Tudo que foi dito até aqui é sobre o dispositivo. Esta seção é sobre a ferramenta. Vocês já escrevem C, mas o C de uma disciplina de algoritmos roda sobre um sistema operacional, uma biblioteca padrão inteira e um `int` de 32 bits. Aqui não há nada disso. As seis diferenças abaixo respondem por quase todos os tropeços do primeiro mês de laboratório — e, ao contrário dos bits de configuração, algumas delas o compilador aceita caladas.

7.1 O programa inteiro, em doze linhas

```
#include <xc.h>
#include <stdint.h>

#define _XTAL_FREQ 16000000UL /* obrigatorio antes de qualquer __delay */

void main(void)
{
    LATD = 0xFF;                /* primeiro o valor do latch */
    TRISD = 0x00;               /* so entao habilita a saida */

    while (1) {                 /* main nao tem para onde retornar */
        LATDbits.LATD0 ^= 1;
        __delay_ms(500);
    }
}
```

Não há `main` que termina, não há `return 0`, não há nada esperando o programa acabar. Esquecido o laço infinito, o processador continua executando o que houver depois do fim de `main` — que é lixo. O `while (1)` não é estilo; é estrutura.

A ordem das duas primeiras linhas é a regra da seção 3 escrita em C: `LAT` antes de `TRIS`.

7.2 `LATD` e `LATDbits` são o mesmo endereço

CONCEITO

O arquivo `xc.h` dá a cada SFR duas faces: o nome do byte inteiro (`LATD`) e uma estrutura de bits (`LATDbits.LATD0` até `LATDbits.LATD7`). Não são registradores diferentes, nem cópias — é o mesmo endereço de memória visto de duas maneiras.

`LATD = 0xFF` altera os oito bits de uma vez. `LATDbits.LATD0 = 1` altera um — mas o processador continua lendo, modificando e reescrevendo o byte inteiro, porque é assim que a memória funciona. Só o compilador esconde isso.

7.3 Ligar um bit sem derrubar os outros

Esta é a operação mais frequente do semestre, e a fonte mais comum de defeitos que não parecem defeitos.

Intenção	Idioma	O que acontece
Ligar o bit n	<code>LATD = (1 << n);</code>	os outros sete permanecem como estavam

Desligar o bit n	<code>LATD &= ~(1 << n);</code>	idem
Inverter o bit n	<code>LATD ^= (1 << n);</code>	idem
Testar o bit n	<code>if (PORTB & (1 << n))</code>	verdadeiro se o bit estiver em 1
Ligar vários	<code>LATD = 0b00001010;</code>	liga RD1 e RD3, preserva o resto

ATENÇÃO

`LATD = 0x01` não liga o bit 0. Ele escreve o byte inteiro: o bit 0 vai a 1 e os outros sete vão a zero. Num roteiro em que RD1 comanda o aquecedor, essa linha desliga o aquecedor — e nada no código diz isso. O sintoma aparece minutos depois, longe da linha alterada.

E `if ((PORTB & 0x10) == 1)` é sempre falso: o resultado do mascaramento é `0x10`, não `1`. Compare com zero, ou não compare com nada.

CONCEITO

Por que existe LAT. Todos os idiomas da tabela são leitura-modificação-escrita: o processador lê o byte, altera um bit e reescreve os oito.

Se a leitura vier de `PORTD`, o que volta é o estado *elétrico* dos pinos. Um pino carregado — um LED, a base de um transistor — pode ler 0 mesmo tendo sido escrito 1. Na reescrita, esse bit vizinho, que estava ligado, é apagado. O código está correto e o hardware desmente a leitura.

Lendo de `LATD`, volta o que foi escrito, e o problema não existe. Daí a regra: **escreva sempre em LAT ; leia PORT apenas quando o pino for entrada.**

7.4 Aqui int tem 16 bits**ATENÇÃO**

No XC8: `char` tem 8 bits, `int` tem 16, `long` tem 32. Um `int` estoura em 32 767. Somar 64 leituras do conversor A/D — $64 \times 1023 = 65\,472$ — já passa disso, silenciosamente, e a média sai negativa. Além disso, o sinal de `char` é decidido pela implementação: não conte com ele.

A regra da disciplina é `#include <stdint.h>` e tipos explícitos: `uint8_t`, `int8_t`, `uint16_t`, `int16_t`. O tamanho passa a estar escrito no código, e não na documentação do compilador.

O tamanho tem custo mensurável: este núcleo é de 8 bits, e cada operação em 16 bits vira duas ou três instruções. Um contador de laço que nunca passa de 200 deve ser `uint8_t` — não por elegância, por ciclos.

CONCEITO

Por que não usamos ponto flutuante. `float` existe, mas é inteiramente emulado em software: uma única multiplicação custa centenas de ciclos e arrasta cerca de 1 KB de Flash — 3% da memória de programa em troca de conveniência de notação.

Daí a convenção que atravessa todo o laboratório: **todas as temperaturas são `int16_t` em décimos de grau.** 25,3 °C é o número 253. E isso sai de graça: o LM35 entrega 10 mV/°C, de modo que a leitura convertida para milivolts já é, numericamente, a temperatura em décimos de grau. Nenhuma divisão, nenhum `float`.

7.5 `__delay_ms` não é uma função**CONCEITO**

`__delay_ms` é uma macro. O compilador a expande num laço de instruções contadas, dimensionado a partir de `_XTAL_FREQ`. Três consequências:

1. o argumento tem de ser constante em tempo de compilação. `__delay_ms(t)` com `t` variável não compila — e é a primeira coisa que vocês vão tentar;
2. `_XTAL_FREQ` errado produz atrasos errados sem nenhum aviso, porque a conta foi feita com a frequência declarada, não com a real;
3. enquanto o atraso corre, o processador não faz absolutamente mais nada. Não lê sensor, não responde a botão, não atende à comunicação.

A terceira consequência é a mais séria, e liga esta seção à seção 5: o tempo é todo de vocês, mas gastá-lo em atraso é gastá-lo. É a razão de existirem os temporizadores e as interrupções, e o Exercício 1.4 mede o tamanho do problema.

7.6 O que não existe aqui

Recurso	Situação no PIC18
<code>printf</code> , <code>scanf</code>	Existem na biblioteca, mas arrastam kilobytes de Flash e uma pilha de argumentos que os 2 KB de RAM não sustentam. A telemetria do Roteiro 8 escreve direto em TXREG
<code>malloc</code> , <code>free</code>	Sem sentido: 2 KB não comportam um <i>heap</i>
Recursão	Compila, mas a pilha de retorno tem 31 níveis em silício
Arquivos, <code>stdin</code> , <code>stdout</code>	Não há sistema operacional para provê-los
<code>main</code> que retorna	Não há para onde retornar
Otimização	A licença gratuita do XC8 compila sem otimizar. O código sai maior e mais lento do que a listagem sugere — o que, para contar ciclos em laboratório, é até conveniente: o que se conta é o que se vê

NOTA

Nada nesta lista é limitação da linguagem. É o C sem a plataforma que normalmente vem embaixo dele — o mesmo padrão, outro contrato com a máquina.

7.7 Do código-fonte ao chip

	Ferramenta	Produto
1	Pré-processador	Código com diretivas resolvidas
2	Compilador	Código de máquina relocável, por módulo
3	Ligador	Programa único, com endereços resolvidos
4	Gerador de imagem	Arquivo <code>.hex</code> , com o conteúdo da Flash
5	Gravador	Dispositivo programado

Dois arquivos produzidos nesse caminho são instrumentos de trabalho, não subprodutos descartáveis. A **listagem** mostra, lado a lado, cada linha de C e as instruções de máquina geradas. O **mapa de memória** informa quanto de Flash e de RAM o projeto consome — os dois orçamentos da seção 4, medidos.

8 Contexto: o resto da arquitetura

Nada nesta seção é necessário para a bancada. Está aqui porque explica **por que** as coisas anteriores são como são, e porque o vocabulário reaparece no seminário comparativo, quando o PIC18 for colocado ao lado de ARM Cortex-M e RISC-V.

CONTEXTO

Contador de programa e vetores. O contador de programa tem 21 bits, capazes de endereçar 2 MB; o dispositivo implementa 32 KB. Três endereços têm significado fixo: `0x0000` é o vetor de reinicialização, `0x0008` o de interrupção de alta prioridade e `0x0018` o de baixa. Entre os dois últimos cabem apenas dezesseis bytes — oito instruções —, o que não comporta um tratador. O que o compilador coloca ali é um desvio para o tratador propriamente dito, alocado em outro ponto.

CONTEXTO

Pilha de hardware. Chamadas de função e interrupções empilham o endereço de retorno numa pilha dedicada de 31 níveis, separada da memória de dados: consumir pilha não consome RAM. Estourar 31 níveis é difícil em código bem estruturado, mas possível com recursão. Diferentemente da família anterior, o PIC18 permite detectar o estouro. Um reset inexplicável em código com recursão tem aí a primeira hipótese.

CONTEXTO

Paralelismo de busca e execução. Como programa e dados usam barramentos distintos, a próxima instrução pode ser buscada enquanto a atual é executada. O processador completa, assim, uma instrução por ciclo. A exceção são os desvios: eles descartam a instrução já buscada e custam dois ciclos — o que explica a segunda linha da tabela de tempos da seção 5.

CONTEXTO

Banqueamento e banco de acesso. Os 2 048 bytes de GPR são organizados em bancos de 256 bytes, porque as instruções não têm bits suficientes para endereçar 2 048 posições diretamente; o endereço completo se forma concatenando o banco selecionado em `BSR` com o deslocamento contido na instrução. Como trocar de banco a cada acesso custa ciclos, o PIC18 abre uma janela virtual de 256 bytes que reúne os 96 primeiros bytes de GPR aos SFR do topo do mapa, e toda instrução tem um bit que seleciona essa janela. O resultado é que variáveis muito usadas e todos os registradores de periférico ficam acessíveis em uma única instrução, sem tocar em `BSR`. Combinado ao endereçamento indireto e ao multiplicador por hardware, é o que torna o PIC18 uma arquitetura **projetada** para receber código de compilador C, e não apenas tolerante a ele.

8.1 Transposição

Quem já viu a família PIC16 reconhecerá quase tudo, com quatro diferenças: o vetor único de interrupção deu lugar a dois, com prioridade; a pilha passou de 8 para 31 níveis e tornou-se observável; o banco de acesso removeu a troca constante de banco; e surgiram o multiplicador por hardware e os modos de endereçamento indireto que viabilizam C eficiente.

Rumo às arquiteturas de 32 bits, três pontos desta aula sobrevivem intactos e um desaparece. Sobrevivem a **árvore de clock** — ainda mais elaborada, com multiplicadores e divisores por periférico, e igualmente capaz de invalidar toda a temporização quando mal configurada —, o **orçamento de memória**, ainda lido no mapa do ligador, e o **acesso a periférico por escrita em endereço**, que continua sendo a forma de comandar um pino em Cortex-M e RISC-V. Desaparece o **banqueamento**: com endereçamento linear de 32 bits, bancos e janelas de acesso deixam de existir, e com eles toda uma classe de erros.

9 Antes de descer para o laboratório

Reserve os últimos dez minutos desta aula para preencher a tabela abaixo. Ela é a mesma da Oficina 1, e é a razão de as duas sessões estarem coladas: vocês preveem agora, com a teoria fresca, e verificam daqui a trinta minutos.

NOTA

Previsão errada não é problema. Previsão **feita depois** do experimento é: ela deixa de ser previsão e vira racionalização. Preencham antes de sair da sala, rubriquem, e levem a folha para a bancada.

Pergunta	Previsão e justificativa	Ru- brica
P1. O que acontece com o cooler no instante do <i>reset</i> , antes de a primeira linha do programa executar?		
P2. Se <code>TRIS</code> for configurado antes de <code>LAT</code> , o comportamento muda? Como?		
P3. Piscando o cooler a 10 Hz, o que você espera ver e ouvir?		
P4. E a 1 kHz?		
P5. Piscar mais rápido faz o cooler girar mais devagar? Justifique.		

BANCADA

Leve para a bancada: esta folha preenchida, um cronômetro e o código de referência do Roteiro 0. Lembrete de segurança que será repetido lá: os pontos **LAMP**, **HEATER** e **COOLER** estão no trilho de 12 V, e nenhuma garra de terra de osciloscópio encosta neles.

CONCEITO

Voltando ao início. A aula abriu afirmando que a arquitetura decide coisas por vocês. Na aula de laboratório, essa afirmação vira observação: o cooler vai partir com um tranco se `TRIS` vier antes de `LAT`, o LED vai errar o ritmo por um fator três se `_XTAL_FREQ` estiver errado, e vocês vão descobrir que não existe meia velocidade com saída digital pura.

Esse último ponto não tem solução dentro do que foi visto hoje — e é intencional. Ele é a pergunta que abre o Roteiro 3.

10 Exercícios

TAREFA

Exercício 1.1. Um projeto usa cristal de 8 MHz e precisa que o processador opere a 24 MHz. Determine os divisores de entrada e de saída necessários, justificando a restrição que fixa a frequência na entrada do multiplicador. Calcule T_{cy} e o tempo de execução de uma rotina de 400 instruções sem desvios.

(b) Refaça o cálculo de T_{cy} para a configuração da bancada, com núcleo a 16 MHz, e diga em que fator os dois resultados diferem.

TAREFA

Exercício 1.2. Um estudante relata que o botão ligado a um pino da porta B não é detectado, embora o mesmo código funcione com o botão movido para a porta D. O programa está correto. Indique a causa mais provável, o bit envolvido e a correção.

TAREFA

Exercício 1.3. Considere um vetor de 64 amostras de 16 bits para média móvel, mais dois buffers de comunicação de 32 bytes cada e uma cadeia de texto constante de 200 caracteres. Calcule o consumo de RAM e o de Flash, indicando em qual memória cada item reside e que fração dos recursos disponíveis é consumida.

TAREFA

Exercício 1.4. Na oficina, o cooler foi acionado por um laço que alterna 500 μ s ligado e 500 μ s desligado usando `__delay_us`.

- (a) Quantos ciclos de instrução, a 16 MHz, correspondem a cada meio período?
- (b) Que fração do tempo de processador esse laço consome?
- (c) Um termostato precisa, no mesmo intervalo, ler o sensor de temperatura e decidir a ação. Explique por que este laço torna isso impossível e o que precisaria mudar na arquitetura do programa.

TAREFA

Exercício 1.5. Um colega escreveu `_XTAL_FREQ` como `48000000UL` e relata que o LED pisca “devagar demais”. Sem acesso à bancada, determine o fator de erro esperado e descreva o experimento mínimo, com cronômetro, que confirma a hipótese.

TAREFA

Exercício 1.6. Num roteiro posterior, RD0 comanda um LED indicador e RD1 comanda o aquecedor, ambos ativos em nível alto. O aquecedor está ligado. Um estudante acrescenta a linha `LATD = 0x01;` para acender o LED.

- (a) Descreva o que acontece com o aquecedor e por quê.
- (b) Escreva a linha correta.
- (c) O compilador emite algum aviso? Justifique em termos do que a linguagem pode e não pode saber.

TAREFA

Exercício 1.7. Um botão está ligado a RB4. O código `if ((PORTB & 0x10) == 1) { ... }` nunca entra no bloco, mesmo com o botão pressionado. Há **duas** causas independentes atuando aqui — uma de linguagem e uma de configuração. Identifique as duas e corrija ambas.

TAREFA

Exercício 1.8. Um filtro de média móvel soma 64 leituras do conversor A/D de 10 bits antes de dividir.

- (a) Qual é o maior valor possível da soma?
- (b) O acumulador foi declarado `int`. Explique o resultado observado.
- (c) Escolha o tipo de `<stdint.h>` adequado, justifique, e diga quantos bytes de RAM o vetor de amostras mais o acumulador consomem — e que fração dos 2 048 bytes isso representa.

TAREFA

Exercício 1.9. Um sistema precisa reagir a um sinal externo em, no máximo, 20 μ s, sempre — não em média.

- (a) O PIC18F4550 a 16 MHz atende? Justifique com os números da seção 5.

- (b) Um computador de mesa, milhares de vezes mais rápido, atende? Justifique.
- (c) Que propriedade distingue os dois casos, e por que ela não se deduz da frequência de clock?

NOTA

A discussão sobre em que sentido “a arquitetura Harvard é mais rápida que a de Von Neumann” é correta, e em que sentido é uma simplificação indevida quando se comparam processadores de gerações diferentes, fica reservada ao **seminário comparativo** — momento em que vocês terão ao lado do PIC18 os dados de um Cortex-M e de um RISC-V para sustentar a comparação.